

Знакомство с OpenMP

часть 2

Балансировка загрузки

- ❑ Важные аспекты производительности
- ❑ Для обычных операций (например, векторное сложение) балансировка редко нужна
- ❑ Для менее регулярных операций требуется балансировка
Примеры:
 - Транспонирование матриц
 - Умножение треугольных матриц
 - Поиск в списке

Разделения работы между потоками

Балансировка загрузки параллельного исполнения цикла **for**
#pragma omp for *дополнительные параметры:*

schedule – планировщик распределения итераций цикла между потоками

schedule(static, chunk) – статическое распределение

schedule(dynamic, chunk) – динамическое распределение

schedule(guided, chunk) – управляемое распределение

schedule(runtime) – определяется OMP_SCHEDULE

nowait – отключение синхронизации в конце цикла

ordered – выполнение итераций в последовательном порядке

collapse(n) – объединить n вложенных циклов в одно

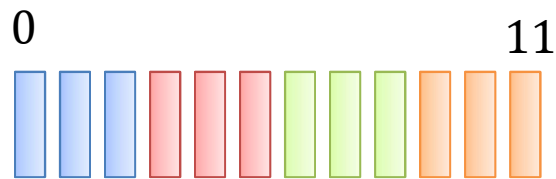
итерационное пространство (доступно с OpenMP 3.0)

Разделение работы между потоками

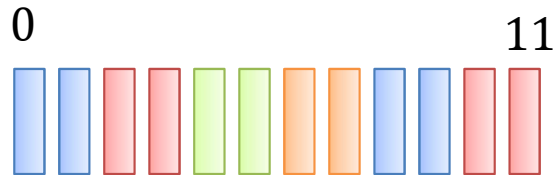
`schedule(static, chunk)` – статическое распределение

Итерации делятся на части, размер которых определен в `chunk`. Части статически заданны для нитей в группе в круговой циклической форме в порядке следования номеров нитей.

По умолчанию размер части: одна непрерывная часть для каждого потока. (пример для четырёх потоков)



`schedule(static)`



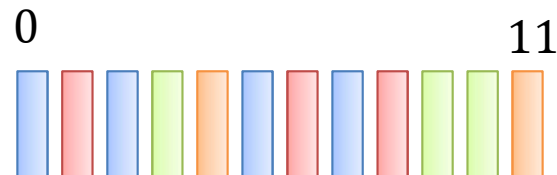
`schedule(static, 2)`

`schedule(dynamic, chunk)` – динамическое распределение

Итерации разбиваются на части, размера `chunk`. Как только каждый поток заканчивает часть, он динамически получает следующую часть.

По умолчанию размер части: 1.

`schedule(dynamic)`

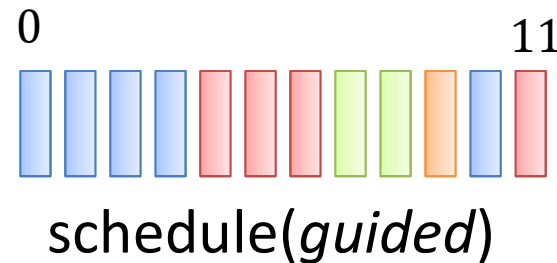


Разделение работы между потоками

schedule(guided, chunk) – управляемое распределение

Размер части уменьшается экспоненциально на каждом итерационном шаге. chunk определяется как наименьшая из возможных частей.

По умолчанию размер части: 1.



schedule(runtime) – планирование определяется через переменную окружения OMP_SCHEDULE

Решение относительно планирования не принимается, пока программа не загружена.

Тип schedule и размер блока может быть выбран на этапе запуска через переменную окружения OMP_SCHEDULE

Разделения работы между потоками

Параметр Ordered пример

```
int i, j, k, n; double x;
#pragma omp parallel for ordered private(n,i,j)
    for (i = 0; i < N; i++)
    {
        k = rand();
        x = 1.0;
        n = omp_get_thread_num();
        for (j = 0; j < k; j++)
            x = sin(x);
        printf("No order : поток %d элемент %d\n", n,i);
#pragma omp ordered
        printf("Order: поток %d элемент %d\n", n, i);
    }
```

Синхронизация потоков

Директивы синхронизации потоков:

➤ *critical*

➤ atomic

➤ barrier

➤ master

➤ single

➤ flush

➤ ordered

Критическая секция

```
int x;  
x = 0;  
  
#pragma omp parallel  
{  
  #pragma omp critical  
  {  
    x = x + 1;  
  }  
}
```

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- *atomic*
- barrier
- master
- single
- flush
- ordered

Атомарная операция

```
int i, index[N], x[M];

#pragma omp parallel for \
    shared(index, x)
    for (i=0; i<N; i++)
    {
#pragma omp atomic
        x[index[i]] += count(i);
    }
```


Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- *barrier*
- master
- single
- flush
- ordered

Барьер

```
int i;  
  
#pragma omp parallel for  
for (i=0;i<1000;i++)  
{  
    printf(" %d",i);  
} // неявный #pragma omp barrier
```

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- *barrier*
- master
- single
- flush
- ordered

Барьер

```
#pragma omp parallel
{
    printf("Hello!\n");
#pragma omp barrier
    printf("I am thread % d\n",
        omp_get_thread_num());
}
```

Синхронизация потоков

barrier, ситуация для применения

Предположим мы выполняем следующий код:

```
for (i=0; i < N; i++)  
    a[i] = b[i] + c[i];  
for (i=0; i < N; i++)  
    d[i] = a[i] + b[i];
```

Если циклы выполнять параллельно, то может быть неправильный ответ

❑ Нужна синхронизация по доступу к **a[i]**

Каждый поток ждет, пока все потоки достигнут определенную точку:

– #pragma omp barrier

Синхронизация потоков

barrier, ситуация для применения

//Кусок кода ...

```
omp_set_num_threads(4);  
#pragma omp parallel  
{  
    int n = 1000;  
    for (int i = 0; i < n; i++)  
    {  
        someFunc();  
    }  
}
```

/* Исполнение кода заблокируется до тех пор, пока все потоки не пройдут цикл */

```
#pragma omp barrier  
    someFunc2();  
}
```

//Кусок кода ...

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- barrier
- *master*
- single
- flush
- ordered

Выполнение кода только главным потоком

```
#pragma omp parallel
{
    //код
#pragma omp master
    {
        // критический код
    }
    // код
}
```

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- barrier
- master
- *single*
- flush
- ordered

Выполнение кода только одним потоком

```
#pragma omp parallel
{
    //код
#pragma omp single
    {
        // критический код
    }
    // код
}
```

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- barrier
- master
- single
- *flush*
- ordered

Согласование значения переменных между потоками

```
int x = 0;
#pragma omp parallel sections \
    shared(x)
{
    #pragma omp section
    { x = 1;
    #pragma omp flush
    }
    #pragma omp section
    {
        while (!x);
    }
}
```

Синхронизация потоков

Директивы синхронизации потоков:

- critical
- atomic
- barrier
- master
- single
- flush
- *ordered*

Выделение упорядоченного блока в цикле

```
#pragma omp parallel for ordered
for (int i = 0; i < N; i++)
{
    int k = rand();
    double x = 1.0;
    for (int j = 0; j < k; j++)
        x = sin(x);
    printf("No order : % d\n", i);
#pragma omp ordered
    printf("Order: % d\n", i);
}
```


Функции блокировки (Lock) в OpenMP

Блокировки – более гибкий способ для управления критическими секциями:

- ❑ Возможно реализовать асинхронное поведение
- ❑ Используются специальные переменные C/C++ типы:
 - omp_lock_t** – простая блокировка (нельзя блокировать дважды)
 - omp_nest_lock_t** – вложенная блокировка (один поток может многократно блокировать переменную перед разблокированием)
- ❑ Можно управлять только через API
- ❑ Без инициализации переменных, поведение функций блокировок не определено

Функции блокировки (Lock) в OpenMP

omp_lock_t (простая блокировка)

Инициализация блокировки

```
void omp_init_lock(omp_lock_t *lock)
```

Уничтожение блокировки

```
void omp_destroy_lock(omp_lock_t *lock)
```

Установка блокировки

```
void omp_set_lock(omp_lock_t *lock)
```

Снятие блокировки

```
void omp_unset_lock(omp_lock_t *lock)
```

Проверка на возможность установки и установка блокировки

```
int omp_test_lock(omp_lock_t *lock)
```

omp_nest_lock_t (вложенная блокировка)

```
void omp_init_nest_lock(omp_nest_lock_t *lock)
```

```
void omp_destroy_nest_lock(omp_nest_lock_t *lock)
```

```
void omp_set_nest_lock(omp_nest_lock_t *lock)
```

```
void omp_unset_nest_lock(omp_nest_lock_t *lock)
```

```
int omp_test_nest_lock(omp_nest_lock_t *lock)
```

Функции блокировки (Lock) в OpenMP

Пример использования блокировок

```
. . . . .
int main()
{
    int i,max;
    int x[1000];
    omp_lock_t lock; //объявление
    omp_init_lock(&lock); //инициализация
    for (i=0;i<1000;i++) x[i]=rand();
    max = x[0];
    #pragma omp parallel for shared(x,lock)
    {
        for (i=0;i<1000;i++)
        {
            omp_set_lock(&lock); //установка замка (блокировки)
            if (x[i]>max) max=x[i];
            omp_set_unlock(&lock); //снятие замка (блокировки)
        }
    }
    omp_destroy_lock(&lock); //уничтожение
    return 0;
}
```

Понятие задачи. Директива task

Явные задачи (explicit tasks) задаются при помощи директивы:

#pragma omp task [опция [[,] опция] ...]

структурный блок

где опция одна из :

- ☐ if (*expression*)
- ☐ untied
- ☐ shared (*list*)
- ☐ private (*list*)
- ☐ firstprivate (*list*)
- ☐ default(shared | none)

В результате выполнения директивы task создается новая задача, которая состоит из операторов структурного блока; все используемые в операторах переменные могут быть локализованы внутри задачи при помощи соответствующих опций. Созданная задача будет выполнена одним потоком из группы.

Для синхронизации задач используется директива **taskwait**

Доступно с OpenMP 3.0

© Зотин Александрович, к.т.н. доцент каф. ИВТ, СибГУ им. М.Ф. Решетнева

Понятие задачи. Директива task

Пример использования

```
typedef struct node node;
struct node {
    int data;
    node * next;
};
void increment_list_items(node * head)
{
    #pragma omp parallel
    {
        #pragma omp single
        {
            node * p = head;
            while (p) {
                #pragma omp taskfirstprivate (p)
                process(p);
                p = p->next;
            }
        }
    }
}
```

Понятие задачи. Директива task

Пример использования 2

```
int fib(int n)
{
    int i, j;
    if (n < 2)
        return n;
    else
    {
        #pragma omp task shared(i)
            i = fib(n - 1);
        #pragma omp task shared(j)
            j = fib(n - 2);
        #pragma omp taskwait
            return i + j;
    }
}
```

Более реальным примером является сортировка qsort

Понятие задачи. Директива task

Пример использования с опцией if

```
int main()
{
double* item;
#pragma omp parallel shared (item)
{
#pragma omp single
{
    int i, size;
    scanf("%d", &size);
    item = (double*)malloc(sizeof(double) * size);
    for (i = 0; i < size; i++)
#pragma omp task if (size > 10)
        process(item[i]);
    }
}
}
```

Если накладные расходы на организацию задач превосходят время, необходимое для выполнения блока операторов этой задачи, то блок операторов будет немедленно выполнен потоком, выполнившим директиву task.

Понятие задачи. Директива task

Пример использования с опцией untied

```
#define LARGE_NUMBER 100000000
double item[LARGE_NUMBER];
extern void process(double);

int main()
{
    #pragma omp parallel
    {
        #pragma omp single
        {
            int i;
            #pragma omp task untied
            {
                for (i=0; i<LARGE_NUMBER; i++)
                    #pragma omp task
                    process(item[i]);
            }
        }
    }
}
```

Опция untied - выполнение задачи после приостановки может быть продолжено любой нитью группы

Наиболее часто встречаемые проблемы

Взаимная блокировка директива critical

```
. . . . .
int A[N], B[N], sum;
#pragma omp parallel num_threads(10)
{
    int iam=omp_get_thread_num();
    if (iam ==0) {
        #pragma omp critical (update_a)
        #pragma omp critical (update_b)
        sum +=A[iam];
    } else {
        #pragma omp critical (update_b)
        #pragma omp critical (update_a)
        sum +=B[iam];
    }
}
. . . . .
```

Наиболее часто встречаемые проблемы

Ошибки установки замков (Lock)

```
#define Max(a,b) ((a)>(b)?(a):(b))
int main ()
{
    omp_lock_t lck;
    float A[N], maxval;

    #pragma omp parallel
    {
        #pragma omp master
        maxval = 0.0;
        #pragma omp barrier
        #pragma omp for
        for(int i=0; i<N;i++) {
            omp_set_lock(&lck);
            maxval = Max(A[i],maxval);
            omp_unset_lock(&lck);
        }
    }

    return 0;
}
```

Наиболее часто встречаемые проблемы

Ошибки установки замков (Lock) исправление

```
#define Max(a,b) ((a)>(b)?(a):(b))
int main ()
{
    omp_lock_t lck;
    float A[N], maxval;
    omp_init_lock(&lck);
    #pragma omp parallel
    {
        #pragma omp master
        maxval = 0.0;
        #pragma omp barrier
        #pragma omp for
        for(int i=0; i<N;i++) {
            omp_set_lock(&lck);
            maxval = Max(A[i],maxval);
            omp_unset_lock(&lck);
        }
    }
    omp_destroy_lock(&lck);
    return 0;
}
```

Наиболее часто встречаемые проблемы

Взаимная блокировка Lock (вариант 1)

```
. . . . .  
#pragma omp parallel  
{  
    int iam=omp_get_thread_num();  
    if (iam ==0) {  
        omp_set_lock (&lcka);  
        omp_set_lock (&lckb);  
        x = x + 1;  
        omp_unset_lock (&lckb);  
        omp_unset_lock (&lcka);  
    } else {  
        omp_set_lock (&lckb);  
        omp_set_lock (&lcka);  
        x = x + 2;  
        omp_unset_lock (&lcka);  
        omp_unset_lock (&lckb);  
    }  
}
```

.

Наиболее часто встречаемые проблемы

Взаимная блокировка Lock (вариант 2)

```
. . . . .  
#pragma omp parallel  
{  
    int iam=omp_get_thread_num();  
    if (iam ==0) {  
        omp_set_lock (&lcka);  
        while (x<0);  
        /*цикл ожидания*/  
        omp_unset_lock (&lcka);  
    } else {  
        omp_set_lock (&lcka);  
        x++;  
        omp_unset_lock (&lcka);  
    }  
}  
}
```

Наиболее часто встречаемые проблемы

Неинициализированные переменные, ошибка

```
#define N 100
#define Max(a,b) ((a)>(b)?(a):(b))

. . . .

float A[N], maxval, localmaxval;
maxval = localmaxval = 0.0;
#pragma omp parallel private (localmaxval)
{
    #pragma omp for
    for(int i=0; i<N;i++) {
        localmaxval = Max(A[i],localmaxval);
    }
    #pragma omp critical
    maxval = Max(localmaxval,maxval);
}

. . . .
```

Наиболее часто встречаемые проблемы

Неинициализированные переменные, исправление

```
#define N 100
#define Max(a,b) ((a)>(b)?(a):(b))

. . . .

float A[N], maxval, localmaxval;
maxval = localmaxval = 0.0;
#pragma omp parallel firstprivate (localmaxval)
{
    #pragma omp for
    for(int i=0; i<N;i++) {
        localmaxval = Max(A[i],localmaxval);
    }
    #pragma omp critical
    maxval = Max(localmaxval,maxval);
}

. . . .
```

Наиболее часто встречаемые проблемы

Неинициализированные переменные вариант 2

```
static int counter;  
#pragma omp threadprivate(counter)
```

```
int main ()  
{  
    counter = 0;  
    #pragma omp parallel  
    {  
        counter++;  
    }  
}
```


Наиболее часто встречаемые проблемы

Неинициализированные переменные вариант 2

```
static int counter;  
#pragma omp threadprivate(counter)  
  
int main ()  
{  
    counter = 0;  
    #pragma omp parallel copyin (counter)  
    {  
        counter++;  
    }  
}
```

Наиболее часто встречаемые проблемы

Зависимости данных и гонки в циклах

Ошибочный вариант реализации

```
#pragma omp parallel for  
for (I=1; I<N; I++)  
    a[I] = a[I-1] + heavy_func(I);
```

Корректная реализация

```
#pragma omp parallel for  
for (I=1; I<N; I++)  
    a[I] = heavy_func(I);
```

// последовательное выполнение

```
for (I=1; I<N; I++)  
    a[I] += a[I-1];
```